

ShinySwarm: Harnessing Microservices and Containers for Collaborative R-Shiny Applications

Iva Gunjaca

14th July 2025

1 Project details

Project title: A cloud-based, microservice-oriented system for enabling real-time, multi-user collaboration and reproducibility in containerised R Shiny web applications.

Student Name: Iva Gunjaca

Host organisation or research group: The MultiScale Networked Systems group at Informatics Institute at the University of Amsterdam.

Contact person: Dr Zhiming Zhao, group leader at the MultiScale Networked Systems, +31638560996, Z.Zhao@uva.nl

2 Project summary

The R programming language [29] has become a mainstay in the scientific community for conducting statistical analysis and facilitating high-quality data visualisation. Despite its power, utilising R requires a degree of programming literacy that remains a significant barrier for many domain experts. To mitigate this, the Shiny package [22] for R offers a paradigm where interactive web applications can be created using only R, without the need for web development skills such as HTML, CSS, or JavaScript. This has enabled statisticians and scientists to construct user-friendly graphical interfaces, thereby broadening accessibility and enhancing the utility of advanced analytics to users lacking formal programming training [15].

Nature's survey of 1,576 researchers has shown that more than 70% of experiments presented in research papers cannot be reproduced by independent peers, and alarmingly, more than 50% of experiments are not even reproducible by the original authors themselves [11]. According to Peng [21], lack of reproducibility and false positive results

can often be attributed to poorly done data analysis that is not available to the public. That is, once the code and the data are available to the public, errors and issues in the research can be easily identified.

While Shiny [22] applications simplify the process of deploying interactive analyses, they are typically architected for single-user, isolated interactions. In contrast, modern scientific discovery is inherently collaborative, relying on sharing, iterative improvement, and collective review [17]. Current Shiny [22] deployments provide limited support for real-time, multi-user collaboration, thereby constraining the ability of research teams to collectively explore data, reproduce analytical workflows, or transparently communicate methodological choices.

Containerisation, using tools such as Docker [10], has emerged as a promising solution to issues of reproducibility and code reusability. By encapsulating the entire computational environment, including code, dependencies, and system libraries, containerisation ensures that analyses can be reliably re-executed and shared across different systems and collaborators. Moreover, containerisation facilitates the deployment and scaling of applications in cloud environments, a crucial aspect for enabling collaborative access and interaction.

Despite these capabilities, the collaborative potential of Shiny [22] applications remains underutilised. Existing approaches to collaboration, such as reactive databases [2] or URL-based state sharing [1] [7], are limited in scalability and often lack robustness or have been deprecated. There is a pressing need for architectural innovations that can support diverse modes of multi-user interaction, such as observing, editing, and synchronising analytical states in real time, without compromising performance or reproducibility.

To address these gaps, this work explores a new architecture for collaborative scientific computing: the design and deployment of containerised R Shiny applications as cloud-based, microservice-oriented systems. By leveraging containerization and message brokers (e.g., Apache Kafka [14]), we aim to enable real-time, multi-user collaboration, seamless cloud deployment, and strict reproducibility through the encapsulation of computational environments and workflows.

To demonstrate these concepts, we apply our approach to a concrete scientific use case: BioDT Honeybee pDT / Pollinators UC [5] Shiny web application, which supports the analysis of honeybee health, behaviour, and population dynamics. This case represents a common challenge in ecology and environmental science, where researchers must collaborate across institutions to analyse shared data, visualise complex models, and ensure transparency and reproducibility. By modularising the core analytic and interface components of the honeybee application, containerising them, and deploying within a cloud-based event-driven architecture, we will evaluate collaborative modes, such as live data exploration, shared state management, and reproducible workflow persistence, using quantitative metrics on usability, performance, and scalability.

In summary, we seek to contribute a lightweight, scalable, and truly collaborative computational research environment for the scientific community. This will add a crucial new dimension to tools like R Shiny [22] and provide actionable patterns for supporting open, reproducible, and collaborative science.

RQ	Research Question	Focus/Outcome
1	How can R Shiny web applications be architected for real-time, multi-user collaboration in cloud environments while maintaining reproducibility and scalability?	System architecture and requirements for collaborative Shiny apps.
2	What are the trade-offs between different event-driven microservice architectures for collaborative R Shiny applications?	Performance, scalability, and usability of architecture alternatives.
3	How effective is the proposed approach in supporting domain-specific collaborative workflows (e.g., honeybee research)?	Demonstration and evaluation via use case in a scientific context.
4	What challenges arise in implementing this approach, and how can they be addressed?	Practical barriers and proposed solutions/best practices

Figure 1: Research Questions and their Focus

3 Problem analysis

The first question to ask is what it means for an RShiny web application to be collaborative. According to Churchill et al [8], collaborative virtual environments (CVE-s) can be defined as “distributed virtual reality systems graphically realised, potentially infinite, digital landscapes.”. These digital landscapes provide potentially limitless virtual spaces where users can engage in information sharing. Individuals can interact both independently and collaboratively with data representations, fostering an immersive platform for communication and collaboration among participants [8]. Furthermore, CVEs (Collaborative Virtual Environments) are developed with specific objectives [8]. In this context, the primary function of a CVE is to facilitate the sharing and usage of Shiny web applications via a web browser interface, allowing users to engage with applications both individually and collaboratively. Interaction with Shiny web applications can be classified as:

- Users can view the Shiny web application and receive some kind of knowledge from its visualisations, that is, the user can “read” Shiny web applications.
- Users can interact with Shiny web applications and with their interaction change the state of Shiny web applications, that is the user can “write” into Shiny web applications.

Collaborative interactions with Shiny web applications can be further classified into:

- Read-read collaboration, where multiple users concurrently acquire information from the application.
- Write-write collaboration, where multiple users simultaneously alter the application state.
- Read-write collaboration is a hybrid approach in which all users can ‘read’ while some can ‘write’ within the application.
- Users can interact individually with the same Shiny web application without interference from others.
- Users can share the current state of the Shiny web application with other users.

3.1 State Sharing

Shiny web applications are designed to generate specific outputs based on user inputs, enabling users to share their findings by replicating the original inputs to achieve identical results. The “shinyURL” [1] package and bookmarking in Shiny applications [7] support this process by encoding the user’s input values into a URL, facilitating state sharing between users. Additionally, bookmarking in Shiny applications [7] offers encoding of the input values in an external file. However, this approach requires double the computational power, as subsequent users must recompute the outputs that the initial user already processed. Ideally, the system would transfer both inputs and outputs: inputs would allow the receiving user to understand how the original user derived the output, while outputs would prevent the need for redundant calculations. Based on this reasoning, two functional requirements have been formulated: the system must transfer inputs of the web application from one user to another, and it should transfer outputs to optimise computational efficiency.

3.2 Read-Read, Read-Write, Write-Write Collaboration

Shiny currently supports read-read collaboration, where multiple users can access and observe the application simultaneously to gain insights. This process is straightforward, as it involves both users simply opening the same app to view its content. The write-write type of collaboration can also be implemented by designating input and output variables as public and providing all users access through a shared hyperlink. However, there is currently no publicly available research that addresses potential challenges or issues that may arise with this method.

Write-write collaboration has been explored previously with the R package “shiny.collections.” [2] This package offers real-time collaborative features akin to “Google Docs-like live collaboration” [2] within Shiny applications, allowing multiple users to concurrently interact and modify the application’s state together. The package was presented at UseR!2017 conference in Brussels [2] and gained a lot of interest. Unfortunately, the package is currently deprecated and no longer supported [2].

The collaboration was facilitated by implementing a shared reactive database accessible to all collaborating users, which stored output data. A Shiny web application

then retrieved this output from the database and displayed it to all users in real time. Because Shiny is a reactive framework that requires a signal to refresh the display whenever the output changes, the use of a reactive database was an ideal solution. When the output was updated, the database would trigger a signal, prompting Shiny to update the interface for all users. [2]

The package is now deprecated because RethinkDB [24], the reactive database it relied on, is no longer supported for use with R. However, this method could be reproduced using another reactive database that is compatible with R. Additionally, the approach could be adapted to support read-write collaboration by restricting read-only users from performing write operations on the database.

Although this approach is effective and can be replicated, challenges emerge when a large number of users engage in write-write collaboration. This scenario is similar to what occurs in a microservices architecture, where multiple services access a shared database. Relying on a shared database introduces several risks, including data corruption, since multiple services are performing concurrent read and write operations. Additionally, a bug in one service could unintentionally alter or delete critical data, potentially affecting all other services that depend on the same database. [25]

Otherwise, there has been no publicly documented exploration of write-write or read-write collaboration. It is possible that such approaches are being investigated internally within companies or in private projects, but there is currently no available information on these efforts.

3.3 Deployment of Shiny Web Applications

Like many web applications, Shiny web applications can be deployed online to enable users to access and interact with them remotely. While Shiny apps can be hosted using standard web deployment methods, there are also several platforms specifically designed to simplify and optimise the deployment of Shiny applications. These include shinyapps.io [23], Shiny Server Open Source [27], and ShinyProxy [28].

3.3.1 shinyapps.io

One of the most popular and user-friendly ways to deploy R Shiny applications is through shinyapps.io, a cloud-based platform provided by Posit (formerly RStudio). Shinyapps.io offers a fully managed hosting service, abstracting away all infrastructure concerns from the developer. From an architectural perspective, the platform operates as a multi-tenant cloud service where Shiny apps run within isolated R processes on backend servers managed by Posit. It provides automatic scaling by dynamically allocating R processes based on incoming traffic, load balancing user sessions across servers, and handling container orchestration behind the scenes [23].

Developers deploy apps directly from RStudio or via command-line tools, while shinyapps.io manages session management, resource allocation, and security. This architecture enables quick deployment, automatic scalability, and ease of maintenance, but offers limited customisation of the underlying runtime environment. Pricing tiers

include a free plan with usage limits and paid plans offering higher performance, custom domains, authentication, and additional security features [23].

3.3.2 Shiny Server Open Source

Shiny Server Open Source is a self-hosted platform that organisations can deploy on their own Linux infrastructure. Architecturally, it acts as a lightweight web server specialised in serving Shiny applications by spawning R sessions for each user connection. When a user accesses an app, Shiny Server initiates a corresponding R process dedicated to that session. It manages HTTP requests, routes traffic to the appropriate app, and serves the UI to the client browser [27].

Unlike shinyapps.io, Shiny Server Open Source does not provide built-in load balancing or advanced authentication. The responsibility for server maintenance, scaling, and security lies with the administrator. Shiny Server supports concurrent users by running multiple R processes, but does not implement containerization or distributed scaling natively [27]. This makes it suitable for smaller-scale deployments or internal applications where full control over the server environment is preferred.

3.3.3 ShinyProxy

ShinyProxy is an open-source platform designed with scalability and security in mind, particularly for containerised Shiny applications. Its architecture is centred around container orchestration: each user session launches an independent Docker container running the Shiny app. This isolation ensures that individual sessions do not interfere with each other’s state or performance and that resources can be allocated and reclaimed efficiently. [28]

ShinyProxy integrates with enterprise authentication systems such as LDAP or Active Directory, enabling fine-grained user access control. It supports horizontal scaling by managing containers across clusters, often leveraging Docker Swarm or Kubernetes for orchestration. This architecture not only enhances security but also allows large-scale deployments to handle many simultaneous users by dynamically launching containers based on demand. [28]

3.4 Microservice Architecture and Shiny

RShiny apps are typically developed and deployed using a monolithic architecture. However, as these applications increase in complexity and size, there has been a growing interest in implementing RShiny apps within a microservices architecture, although this approach remains relatively uncommon. Two examples of R Shiny applications implemented using microservice architecture are the “Shiny Microservices Demo” presented at the useR! 2018 Conference [6] and “Turning our monolithic Shiny app into a microservices-based structure” by Juan Cruz Rodriguez [3]. “Shiny Microservice Demo” presented at the useR! 2018 project can only be found on GitHub, and it does not contain much documentation about the specific approach. From code observation, it seems that they used a similar approach to “Turning our monolithic Shiny app into

a microservices-based structure” by Juan Cruz Rodriguez. In both cases, they use microservice architecture and an API gateway for communication between services. Additionally, Rodriguez uses the “plumber” [26] R package to create a web API from existing R source code with roxygen2-like comments.

Another approach for microservice communication within the R language is through a message broker with “rkafka” [13] package. This package allows for sending and receiving messages over Apache Kafka [14] message broker.

Apache Kafka is an open-source platform that helps manage and process data streams in real-time. It was first created by LinkedIn and is now maintained by the Apache Software Foundation. Kafka is designed to efficiently handle large amounts of data and ensure reliable and scalable messaging. [14]

Kafka works like a large, distributed journal. It allows data to be sent and received using a publish-subscribe model, meaning that data producers send information, and consumers subscribe to get this information. This setup lets Kafka handle a large amount of data quickly and reliably, making it perfect for tasks like collecting logs, processing data as it comes in, merging data from different sources, and running real-time data analysis [14]. In our example case [12], Kafka serves as a message board between services. It facilitates communication by allowing different services to post messages and receive updates seamlessly, ensuring that data flows smoothly and efficiently across the system.

The “rkafka” [13] package has been removed from CRAN and is only available for download from GitHub. It has not been updated since 2022, and it is not recommended to use it with the newest R version. Usage of “rkafka” on GitHub is sparse, and only a few projects can be found. One that is worth mentioning is a tutorial on how to use “rkafka” to connect to Apache Kafka message broker from Confluent by Patrick Neff [19], the second one is a GitHub project of examples of using “rkafka” package [9]. In no scenarios were they using “rkafka” with Shiny. However, author was able to build a simple Shiny web application that adds two numbers, following a microservice architecture [12]. The application was containerised using Docker, with inter-service communication handled via the Apache Kafka message broker. The application can be found publicly on authors GitHub [12]. The issue encountered was that conversion from a dataframe to a JSON message takes up to 5 to 8 seconds, which slows down the application to the point that end-to-end latency takes on average 21 seconds on my local machine.

Neff [19] suggests an alternative approach for integrating Apache Kafka with R by utilising a MongoDB [18] database. In this setup, all messages from the Kafka topic are stored in MongoDB, and the R file reads the messages directly from the database. However, this method is not suitable for our purposes, as it only supports message consumption and does not address our broader requirements.

In 2024, INWT and Andreas Neudecker introduced a new R package for sending and receiving messages from Apache Kafka, simply named “kafka” [20]. They provide a tutorial demonstrating how to use this package with Shiny web applications, indicating that it offers better support for Shiny than the earlier “rkafka” package. While author has not personally implemented, tested, or measured this package, so the author cannot confirm whether the known JSON issue persists, the absence of any mention of such

problems in their tutorial suggests it may have been due to an error on authors part. Additionally, the JSON issue can be mitigated by keeping messages small, which helps reduce processing time and minimises potential complications.

3.5 Data Persistency

Ideally, our solution would allow users to return to a Shiny web application and find it in the same state as they left it. While native Shiny does not provide built-in data persistence, there are seven recommended methods for implementing data persistence in Shiny applications [4]. Data persistence can be achieved either locally or on a remote server. Arbitrary data can be stored in files, while structured data is typically saved in tables. These tables can reside in relational databases or be hosted on services like Google Sheets. Examples of remote file storage options include Dropbox and Amazon S3, while relational databases such as SQLite, MySQL, and MongoDB are commonly used for storing structured data. These storage options can be easily accessed and managed through the corresponding R packages [4].

3.6 Logging

No system or application is immune to errors and issues, and our system is no exception. Therefore, it is essential to implement logging mechanisms so that when problems arise, we can quickly detect and address them in a timely manner. Logging involves recording application events such as a user logging in, messages being exchanged between services, or errors occurring. Properly labelling and storing these events is essential, as it enables the management team to effectively analyse log data. Stored information must not contain any sensitive data, such as personal details or passwords, to prevent potential security breaches and protect user privacy.

Apache Kafka [14] naturally stores application events within a structured, centralised message queue, providing an accessible log that can be used to visualise application performance and pinpoint issues. However, once the queue reaches its capacity, Kafka removes older events to make room for new ones. In high-traffic situations, this can result in the loss of important events, which might prevent the maintenance team from identifying and addressing critical errors or issues within the system. Additionally, Apache Kafka does not capture errors or issues that occur within the container's code. While Kafka provides visibility into application events, it should not be relied upon as the sole logging tool in our system.

3.7 User Interface and Authentication

To effectively use our system, users will need an accessible user interface and a robust authentication system. To better understand the needs of future users, I conducted interviews with students at the University of Amsterdam who have a background in biology, as the platform's intended audience includes researchers, teachers, and students in this field. From these interviews, the following user stories were identified:

- As a user, I want to log in using my credentials and password so that I can securely access the platform.
- As a user, I want to find uploaded Shiny web applications so that I can quickly locate the apps I need.
- As a user, I want to access uploaded Shiny applications so that I can use their features and functionalities.
- As a user, I want an easy way to invite other users to a collaborative session so that we can work together seamlessly.
- As a user, I want to see which other users are present in the collaborative session so that I know who is participating with me.
- As a user, I want to set and update my avatar and bio so that other users can recognise me and learn more about my background.
- As an admin, I want to manage user permissions and Shiny applications so that I can maintain security and oversee platform operations.
- As a user, I want to mark applications as favourites so that I can quickly access the items I use most often.
- As a user with accessibility needs, I want the platform to provide features such as screen reader compatibility and keyboard navigation so that I can use all functionalities with ease.
- As a user, I want the option to enable dark mode so that I can reduce eye strain and use the platform comfortably in low-light environments.
- As a user, I want to see relevant information (e.g., description, usage instructions, creator) for each Shiny application so that I can understand its purpose and how to use it.
- As a user, I want to designate my uploaded Shiny applications as public or private so that I can control who has access to them.

However, a thorough discussion with stakeholders will be essential to ensure that all necessary features are identified and to confirm whether these user stories align with their expectations and vision for the system.

4 Research method

Based on the analysis described previously, the following architecture has been selected: The Shiny UI and server components will be containerised separately from the calculation functions. For example, in a simple calculator web app, the Shiny component will reside in its own container, while operations will be handled in another dedicated

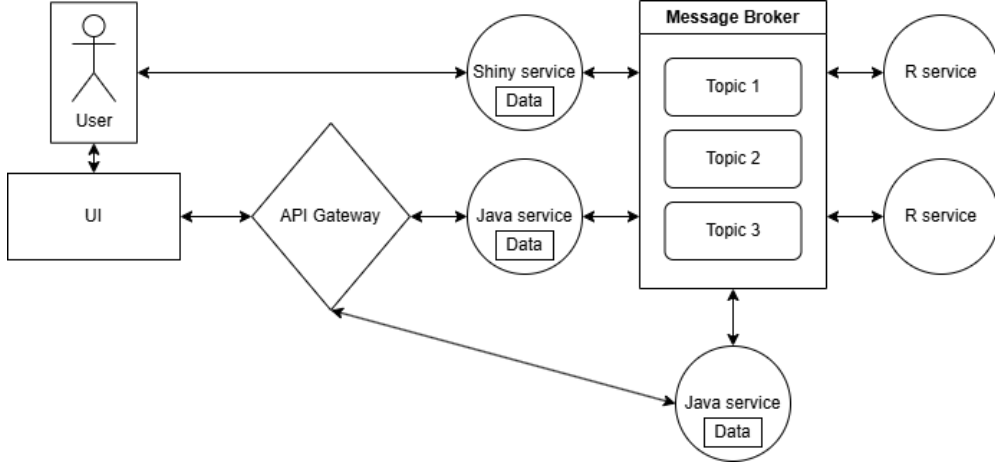


Figure 2: System Architecture: One User

container. Communication between these containers will utilise a Kafka message queue, following an Event-Driven Microservice Architecture. The R "kafka" [20] package will be used for this communication, as it demonstrated the best performance during the evaluation phase. Both the Shiny app and the calculation service will be containerised using Docker. For deployment, these containers will be orchestrated and managed using Kubernetes, ensuring scalability and high availability. A CI/CD pipeline will be set up on GitHub Actions to automate testing, building, and deployment processes, adhering to DevOps best practices. This architecture enables a robust, cloud-based Shiny application that is accessible to users via any web browser, offering scalability, maintainability, and ease of access.

When a user submits input values through the RShiny web application, the data is not processed directly within the Shiny app itself. Instead, the input values are published to a designated Kafka channel to which the calculation containers are subscribed. The appropriate calculation container then consumes the message, performs the necessary computation, and produces a response message containing the output. This output message is subsequently consumed by the Shiny UI container, which updates the user interface to display the results.

Since Kafka messages are structured as key-value pairs, each user will be assigned a unique key upon connecting to the web application. This key ensures that messages in the Kafka queue can be accurately associated with the corresponding user, preventing any mix-ups when multiple users are interacting with the app and sending messages simultaneously. Architecture can be observed in figure 2

To address the collaborative functionality of the environment, two main approaches will be implemented, as illustrated in figures 3 and 4. Both methods will be thoroughly evaluated using state-of-the-art metrics to identify and compare their respective strengths and weaknesses. The objective is to determine the advantages and disadvant-

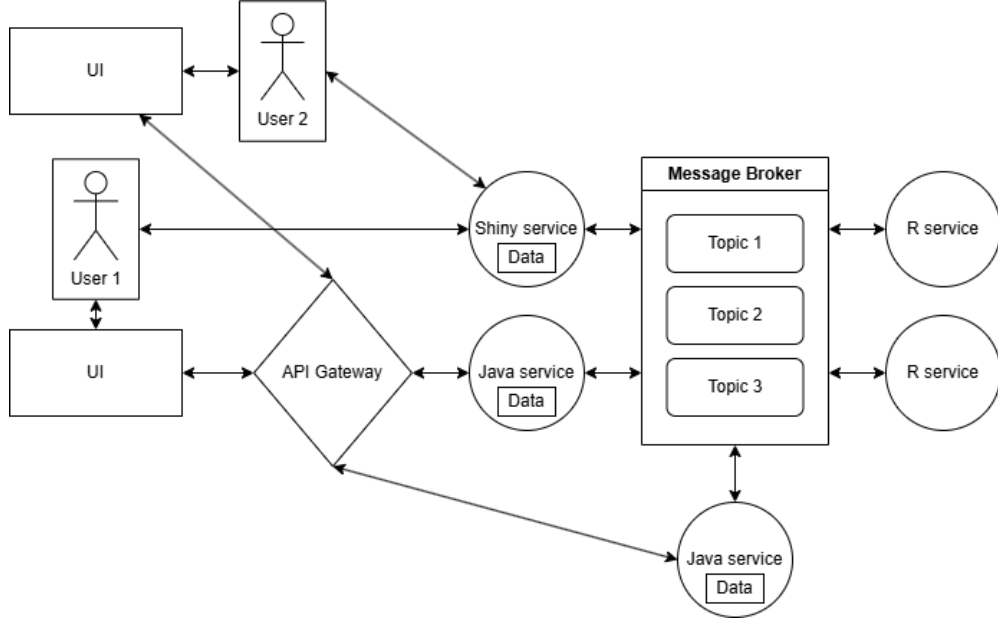


Figure 3: System Architecture: Collaborative Option One

ages of each approach in supporting collaborative features within the system.

In the initial approach outlined in figure 3, a single container will be utilised for the Shiny UI, allowing multiple users to connect and interact with the application simultaneously. This setup facilitates a read-read style of collaboration among users. To enable a write-write collaboration model, all input and output variables need to be designated as public variables within the container. Meanwhile, a read-write collaboration approach can be implemented by assigning specific read and write permissions to users.

In the second approach depicted in figure 4, each user is provided with their own container for the Shiny UI. A unique key is shared among all users participating in the current collaborative environment, enabling write-write collaboration, as each Shiny UI container will process messages tagged with the collaborative group's key. To facilitate read-write collaboration, users with read-only access within the group are restricted from posting input value messages and can solely consume output messages. This means that read-only users are subscribed exclusively to the output topic, not the input topic. To allow visibility of input values by write users, these values must be included in the output messages. However, this could lead to performance issues due to the increase in message size.

To enable state sharing in the Shiny web application, a message containing the recipient user's key along with the input and output values will be sent to the designated Kafka topic. The recipient user will then consume this message, allowing them to view

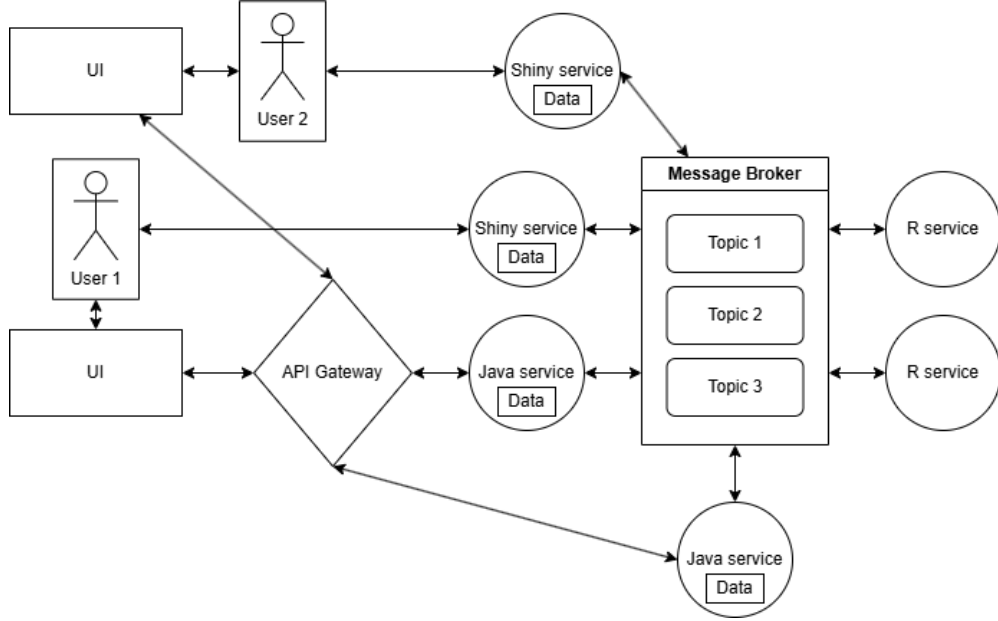


Figure 4: System Architecture: Collaborative Option Two

the state of the original user’s Shiny web application. A limitation of this method is that it requires both users to be online and actively connected to the Kafka queue.

This approach encounters a challenge regarding data persistence, as it only allows users to start a new, unedited session in the Shiny web application, without preserving any data. As a result, if a user exits the application, they cannot resume their progress later. To address this, a database could be implemented to store all input and output values, using the application ID and user ID to reload the user’s last state. This solution could also enable state sharing with offline users.

The database for the system will be implemented using SQLite, chosen for its simplicity and built-in support for authentication. This lightweight database solution is well-suited for rapid prototyping and ensures straightforward integration with the application.

To maintain system reliability and facilitate troubleshooting, comprehensive logging mechanisms will be implemented. These logs will allow for efficient monitoring of the system, providing valuable data to identify, analyse, and resolve any errors or issues that may arise during operation.

The application will be deployed on a private server at the University of Amsterdam (UvA). Deployment, orchestration, and management of the system will be handled using Kubernetes [16], which provides robust tools for scaling, automation, and resource management, ensuring the platform remains stable and flexible as demand evolves.

The user interface (UI) will be web-based, developed using standard web technologies

such as HTML, CSS, and JavaScript, which allow for cross-platform accessibility and a responsive user experience. Any necessary backend development will be done in Java, chosen for its maturity, performance, and excellent support for integration with Apache Kafka, which will be used to facilitate event-driven communication within the microservices architecture.

After developing a working prototype of this system using a simple calculator Shiny web application, a use case will be implemented with the BioDT Honeybee pDT / Pollinators UC [5] Shiny web application to demonstrate and evaluate the key features of this environment. Components created by Sandro Boelsz, during his thesis project, will be containerised and modified for compatibility with the proposed environment. Subsequently, the environment will be deployed, and its performance will be assessed using established metrics. This process will demonstrate the feasibility of a collaborative environment where Shiny web applications can be easily uploaded and accessed through a web browser.

5 Expected results of the project

The anticipated outcomes of this thesis project include:

1. A theoretical framework presenting two alternative methodologies for developing and deploying collaborative Shiny web applications within an event-driven microservice architecture.
2. An open-source, prototypical proof-of-concept (PoC) software solution that implements these methodologies, offering a user-friendly interface and enhanced features.
3. An evaluation of the developed PoC using selected metrics.
4. An analysis of the strengths and weaknesses of the proposed approaches.

6 Required expertise for this project

Domain	Level Required	Authors Level
System Architecture	Advanced	Intermediate
R and Shiny	Intermediate	Intermediate
Docker	Intermediate	Basic
Kubernetes	Advanced	Basic
Apache Kafka	Advanced	Intermediate
Java Backend	Intermediate	Intermediate
Web Frontend	Basic	Basic
CI/CD (GitHub Actions)	Basic	Intermediate
Database (SQLite)	Basic	Intermediate
Logging and Monitoring	Intermediate	Basic
Collaborative System Design	Advanced	Intermediate

7 Timeline

The project's timeline is depicted in the PERT chart in figure 5, where squares labelled A through T represent various implementation and evaluation tasks, while those labelled W signify sections of thesis writing. Each square includes three numbers indicating the best-case, most likely, and worst-case scenarios in terms of days required to complete the tasks.

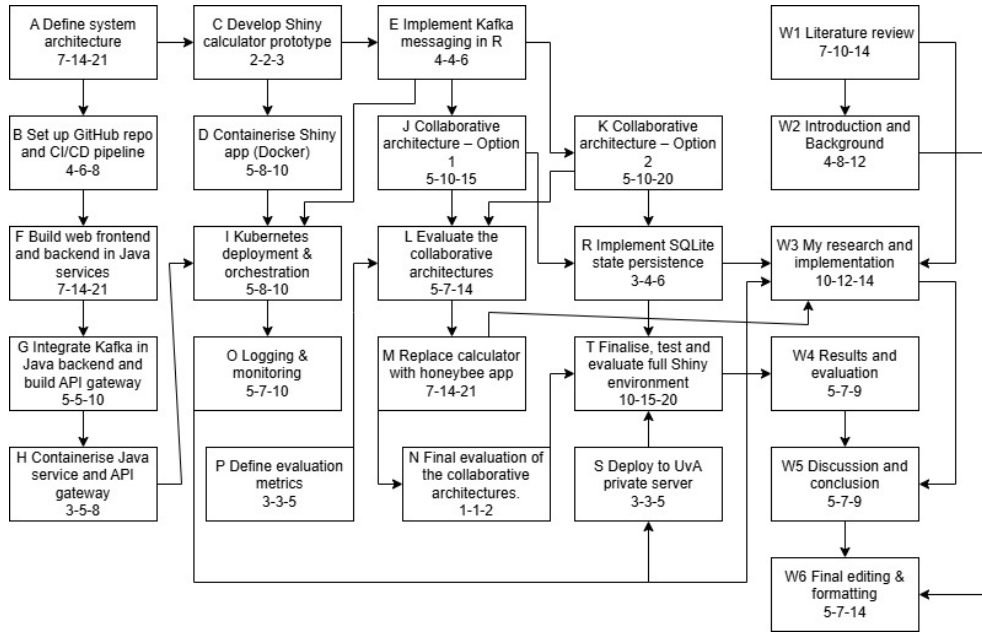


Figure 5: PERT Chart

8 Risks

Given the authors' limited experience with the required technologies and the extensive range of features planned for the user interface, there is a significant risk that not all features will be successfully implemented within the project's timeframe. This lack of familiarity may lead to unforeseen technical challenges and a steeper learning curve, potentially causing delays or the need to scale back on certain functionalities. Additionally, prioritising feature development in an Agile environment may result in some lower-priority features being postponed or excluded from the final proof-of-concept.

9 Literature survey

During this literature review, a range of keywords were employed in Google Scholar, including collaborative environments, Shiny, Shiny web application, Shiny containerisation, Shiny Microservice, Shiny Kafka, R Kafka and Shiny deployment. However, these searches produced limited results, primarily consisting of demonstrations of Shiny web applications. Information regarding the implementation of collaborative environments was particularly scarce. As a result, much of the research had to rely on non-peer-reviewed sources such as miscellaneous articles, GitHub repositories, and YouTube videos.

When Google Scholar failed to provide many relevant articles, I used the same search terms in the regular Google search engine. This allowed me to identify libraries and approaches referenced in the problem analysis. References and their relation to the features of the system can be found in Table 1.

As noted in the previous analysis, the only known example of similar work on collaborative Shiny web applications is “Pipi Popo.” However, their approach is neither containerised nor deployed, and uses a shared database for collaborative features, which could lead to scalability issues.

Traditionally, deployment and containerisation of Shiny web applications follow a monolithic architecture, and there are very few examples utilising a microservices approach. Among the available microservice architecture examples, none demonstrate the use of Apache Kafka or offer dedicated support for collaborative interactions.

The integration of Apache Kafka with Shiny web applications is even less common. In fact, the only example found is within a tutorial on the “kafka” library. This suggests that, while it is possible that someone is developing containerised, microservice-based Shiny web applications using Apache Kafka, I was unable to find any publicly available references or examples of such work.

Based on the literature review, there is no evidence in publicly available sources that a system or deployment approach for Shiny web applications featuring collaborative functionalities, a microservices architecture and utilisation of a message queue has been previously attempted.

10 Bibliography

References

- [1] aoles. *GitHub - aoles/shinyURL: :link: Save and restore the state of Shiny app’s widgets by encoding them in an URL query string*. 2016. URL: <https://github.com/aoles/shinyURL>.
- [2] Appsilon. *GitHub - Appsilon/shiny.collections: This project is deprecated and no longer supported. (Google Docs-like live collaboration in Shiny)*. 2017. URL: <https://github.com/Appsilon/shiny.collections>.

- [3] Appsilon and J. C. Rodriguez. *Juan Cruz Rodriguez: Turning our monolithic Shiny app into a microservices-based structure*. May 2023. URL: <https://www.youtube.com/watch?v=pzKkdadIgFU>.
- [4] D. Attali. *Shiny - Persistent data storage in Shiny apps*. Nov. 2020. URL: <https://shiny.posit.co/r/articles/build/persistent-data-storage/>.
- [5] BioDT. *GitHub - BioDT/uc-pollinators: Honeybee pDT / pollinators UC main repository*. 2024. URL: <https://github.com/BioDT/uc-pollinators>.
- [6] S. Came. *Demonstrating how to host a secure Shiny app in docker*. July 2018. URL: <https://github.com/scottcame/shiny-docker-demo>.
- [7] W. Chang. *Shiny - Bookmarking state*. Oct. 2019. URL: <https://shiny.posit.co/r/articles/share/bookmarking-state/>.
- [8] E. Churchill and D. Snowdon. ‘Collaborative virtual environments: An introductory review of issues and systems’. In: *Virtual Reality* 3 (Mar. 1998), pp. 3–15. DOI: 10.1007/BF01409793.
- [9] A. Dieman. *GitHub - CodiePP/r_kafkat0cabceaf89105a0ee386371bf867a0269dba8141*. 2021. URL: https://github.com/CodiePP/r_kafka/tree/0cabceaf89105a0ee386371bf867a0269dba8141.
- [10] Docker. *Enterprise Application Container Platform — Docker*. 2024. URL: <https://www.docker.com/>.
- [11] O. E. Gundersen. ‘The Reproducibility Crisis Is Real’. In: *AI Magazine* 41 (Sept. 2020), pp. 103–106. DOI: 10.1609/aimag.v41i3.5318.
- [12] I. Gunjaca. *Collaborative RShiny*. 2025. URL: <https://github.com/IfaPifa/Collaborative-RShiny>.
- [13] S. Gupta. *rkafka—Using Apache “Kafka” Messaging Queue Through “R”*. Aug. 2022. URL: <https://github.com/cran/rkafka>.
- [14] A. Kafka. *Apache Kafka*. 2023. URL: <https://kafka.apache.org/>.
- [15] P. Kasprzak, L. Mitchell, O. Kravchuk and A. Timmins. ‘Six Years of Shiny in Research - Collaborative Development of Web Tools in R’. In: *The R Journal* 12.2 (2021), pp. 155–162. DOI: 10.32614/RJ-2021-004. URL: <https://doi.org/10.32614/RJ-2021-004>.
- [16] Kubernetes. *Production-Grade Container Orchestration*. 2019. URL: <https://kubernetes.io/>.
- [17] S. Lu and J. Zhang. ‘Collaborative Scientific Workflows’. In: *2009 IEEE International Conference on Web Services*. 2009, pp. 527–534. DOI: 10.1109/ICWS.2009.150.
- [18] MongoDB. *MongoDB*. 2024. URL: <https://www.mongodb.com/>.
- [19] P. Neff. *Create a Data Analysis Pipeline with Apache Kafka and RStudio — Confluent — DE*. July 2021. URL: <https://www.confluent.io/de-de/blog/data-analytics-pipeline-with-kafka-and-rstudio/>.
- [20] A. Neudecker. *Introducing the kafka R Package*. 2024. URL: <https://www.inwt-statistics.com/blog/r-package-kafka>.

- [21] R. Peng. ‘The Reproducibility Crisis in Science: A Statistical Counterattack’. In: *Significance* 12 (June 2015). DOI: 10.1111/j.1740-9713.2015.00827.x.
- [22] Posit. *Shiny*. URL: <https://shiny.posit.co/>.
- [23] Posit. *shinyapps.io User Guide*. Mar. 2025. URL: <https://docs.posit.co/shinyapps.io/guide/>.
- [24] *RethinkDB: The open-source database for the realtime web*. URL: <https://rethinkdb.com/>.
- [25] C. Richardson. *Microservices Pattern: Shared database*. 2018. URL: <https://microservices.io/patterns/data/shared-database.html>.
- [26] B. Schloerke, J. Allen, B. Tremblay, F. van Dunné and S. Vandewoude. *An API Generator for R*. Feb. 2025. URL: <https://www.rplumber.io/>.
- [27] *Shiny Server Pro Admin Guide*. 2020. URL: <https://docs.posit.co/shiny-server/>.
- [28] *ShinyProxy*. 2025. URL: <https://shinyproxy.io/>.
- [29] R. C. Team. *R: A Language and Environment for Statistical Computing*. 2025. URL: <https://www.r-project.org/>.